Lab 7

1. Java Packages

1.1 What is a Package?

A **package** in Java is like a **folder** that groups related classes together.

Purpose:

- Organize source code and avoid name conflicts.
- Make code easier to maintain and reuse.
- Control access between classes.

Example:
`com.example.school` → means folders `com/example/school/`

1.2 Declaring a Package

At the top of your Java file, write:

```
package com.example.school;
```

This line must be the first line in the file.

3. Example Project Structure

```
project/
 ├── src/
 │    └── com/
 │        └── example/
 │            └── school/
 │                ├── Student.java
 │                └── Main.java
 └── bin/     ← compiled files will go here
```

1.3 Compile your project

Step1: go to root of project.

```
cd project
```

Step2: compile all java file and output to bin folder.

```
javac -d bin com/example/school/*.java
```

✅ The -d bin option means:

Place the compiled .class files into the ../bin folder,
while keeping the same package folder structure.

Step2 - exmple 2 :

- if there are more than one folder has java files using `/**/*.java` or `$(find src -name "*.java")`

```
# If globstar is enabled:
javac -d out src/**/*.java

# If globstar is not enabled or you want a shell-independent method:
javac -d out $(find src -name "*.java")
```

Enable recursive globbing in the shell

```
shopt globstar        # check current globstar status (off by default)
shopt -s globstar     # enable globstar (turn on recursive ** support)
```

1.4 Run your project

```
cd bin
java com.example.school.Main
```

➤ Importing Classes

You can use classes from other packages using the `import` statement.

Example with import

If you have another package:

```
com/example/app/
  ├── Main.java
com/example/school/
  ├── Student.java
```

```
package com.example.app;
import com.example.school.Student;

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Bob");
        s.showInfo();
    }
}
```

Note:

- The `package` statement must come **before** any `import` statements.
- Only **comments or blank lines** can come before `package`.

2. CLASSPATH

2.1 What is the Classpath?

Classpath = the path (or location) where the **Java Virtual Machine (JVM)** and **compiler (`javac`)** look for `.class` files.

👉 It tells Java **where to find**:

- Your compiled classes
- Imported packages
- External libraries (like `.jar` files)

2.2 Default Classpath

By default, Java uses the **current folder (`.`)** as the classpath.

So if you run:

```
java com.example.school.Main
```

Java starts searching for:

```
com/example/school/Main.class
```

2.3 using classpath -cp (short for -classpath)

If your compiled classes are in another folder (like ./bin),
and you are not inside that folder,
you must tell Java where to find them:

```
java -cp ./bin com.example.school.Main
```

---

3. Default Access modifier (Package-Private)

- No modifier written (just leave it blank)
- Accessible **only within the same package**

Example

```
project/
└── src/
    ├── package1/
    │   ├── Student.java
    │   └── Main.java
    └── package2/
        └── Other.java
```

File 1: `Student.java`

Save this file in `src/package1/Student.java`.

```
package package1;

public class Student {
    String name = "Alice";  // default access
}
```

File 2: `Main.java`

Save this file in `src/package1/Main.java`.

```
package package1;

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.name);
```

```
    }
}
```

File 3: `Other.java`

Save this file in `src/package2/Other.java`.

```
package package2;

import package1.Student;

public class Other {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.name); // error
    }
}
```
